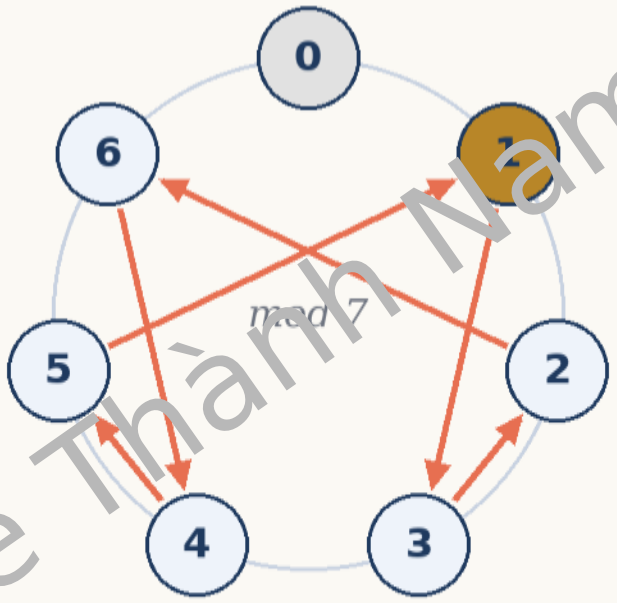
TUYỂN TẬP BÀI TẬP TOÁN TRÊN CODEFORCES

Tập 3 · 16 bài Codeforces, dịch tiếng Việt

SỐ HỌC MODULO



Lê Thành Nam

$3^n \bmod 7$: $1 \to 3 \to 2 \to 6 \to 4 \to 5 \to 1$ (3 là căn nguyên thủy)

Lũy thừa nhanh · Fermat · Euler

Euclid mở rộng · Bézout · Phương trình Diophante

Định lí số dư Trung Hoa · Logarit rời rạc

Ngây thơ → Bế tắc → Phá bế tắc → Tối ưu

LÊ THÀNH NAM

Tổng hợp và biên soạn

MỤC LỤC

Ngây thơ → Bế tắc → Phá bế tắc → Tối ưu

Giới thiệu 2

Chương 1. Lũy thừa nhanh 3

Bài 1. AND bằng 0, tổng lớn *(Codeforces 1514B)* 4
Bài 2. Số nguyên tố và phép nhân *(Codeforces 1228C)* 8
Bài 3. Dãy truy hồi dạng tích *(Codeforces 1182E)* 12

Chương 2. Nghịch đảo modulo, định lí Fermat và định lí Euler 16

Bài 4. Số tuyệt đẹp *(Codeforces 300C)* 17
Bài 5. Robot của ông già Noel *(Codeforces 1279D)* 21
Bài 6. Tích các ước *(Codeforces 615D)* 25
Bài 7. Tháp lũy thừa *(Codeforces 906D)* 29

Chương 3. Euclid mở rộng, Bézout và phương trình Diophante 34

Bài 8. Gỗ mun và ngà voi *(Codeforces 633A)* 35
Bài 9. Mùa giải bóng đá *(Codeforces 1244C)* 39
Bài 10. Đường thẳng *(Codeforces 7C)* 43
Bài 11. Nezzar và tấm bảng *(Codeforces 1477A)* 47

Chương 4. Định lí số dư Trung Hoa 51

Bài 12. Trò chơi số dư *(Codeforces 687B)* 52
Bài 13. Hai cấp số cộng *(Codeforces 710D)* 55
Bài 14. Hai chiếc đèn chùm *(Codeforces 1500B)* 59

Chương 5. Phương trình mũ và logarit rời rạc 64

Bài 15. Phương trình đồng dư *(Codeforces 919E)* 65
Bài 16. Năm mới và dãy truy hồi *(Codeforces 1106F)* 68

Lê Thanh Nam

GIỚI THIỆU

Tài liệu này dành cho ai

Đây là Tập 3 của bộ Tuyển tập bài tập Toán trên Codeforces, gồm 16 bài về số học modulo, sắp thành 5 chương theo kiến thức: *lũy thừa nhanh; nghịch đảo modulo, định lí Fermat nhỏ và định lí Euler; Euclid mở rộng, định lí Bézout và phương trình Diophante tuyến tính; định lí số dư Trung Hoa; phương trình mũ và logarit rời rạc*. Các bài tiêu biểu: 1514B AND 0, Sum Big; 1244C The Football Season; 687B Remainders Game; 1106F Lunar New Year and a Recursive Sequence. Tài liệu phù hợp với học sinh đã học Tập 1 (chia hết, ƯCLN, BCNN) và Tập 2 (số nguyên tố và ước), đang luyện thi học sinh giỏi cấp tỉnh, cấp quốc gia và các kì thi tương đương.

Mỗi bài gồm hai phần. Đề bài được viết lại bằng tiếng Việt, giữ đúng yêu cầu, ràng buộc, định dạng vào/ra và ví dụ của đề gốc, kèm bảng subtask do tài liệu bổ sung. Hướng dẫn giải đi qua 4 bước tư duy; mỗi bước có code chạy được, có hình minh họa, bài nào cũng kết thúc bằng bảng so sánh các tầng lời giải đo thật trên máy. Mọi định lí được dùng đều được phát biểu lại và giải thích vì sao áp dụng được.

4 bước tư duy giải một bài

Bước	Nội dung
Ngây thơ	Ý tưởng tự nhiên nhất, làm đúng như đề mô tả, có code và độ phức tạp.
Bế tắc	Biểu đồ thời gian đo thật, chỉ ra chỗ chậm hoặc chỗ sai, kết thúc bằng một câu hỏi then chốt.
Phá bế tắc	Nhận xét hoặc tính chất giúp vượt qua chỗ tắc, có ví dụ và hình minh họa.
Tối ưu	Code cuối cùng, độ phức tạp và các lưu ý: kiểu dữ liệu, tràn số, đọc ghi nhanh.

Lê Thanh Nam

Subtask và bảng so sánh các tầng

Đề Codeforces không chia subtask. Tài liệu bổ sung cho mỗi bài ba mức: Subtask 1 (30%) là ràng buộc nhỏ để cách ngây thơ lấy điểm, Subtask 2 (30%) và Subtask 3 (40%) là ràng buộc gốc hoặc ràng buộc mở rộng (ví dụ số mũ tới 10^{18}, hệ số tới 10^{12}) để thấy rõ vai trò của từng định lí.

Cuối mỗi bài là bảng so sánh: mỗi tầng code chạy trên test lớn nhất của từng subtask, ghi thời gian thật và số điểm. Đầu mỗi bài ghi giới hạn thời gian gốc của Codeforces; riêng bảng so sánh dùng chung một giới hạn 1 giây cho mọi bài để dễ so sánh. Một tầng chỉ được điểm của subtask nếu cho kết quả đúng trong không quá 1 giây, dùng không quá 512 MB bộ nhớ và 8 MB ngăn xếp (ô "sai" là chương trình cho kết quả sai, ô "lỗi" là chương trình bị dừng vì tràn bộ nhớ hoặc tràn ngăn xếp). Thời gian đo bằng g++ 13 (tùy chọn -O2) trên máy chủ Linux, tính cả đọc và ghi dữ liệu, lấy lần nhanh hơn trong hai lần chạy.

Sử dụng tài liệu

Với mỗi bài, hãy tự suy nghĩ ít nhất 20–30 phút trước khi đọc hướng dẫn. Khi đọc, dừng lại ở câu hỏi then chốt cuối phần **Bế tắc** và tự trả lời trước khi xem phần **Phá bế tắc**.

Mọi code trong tài liệu đã được biên dịch, chạy đúng mọi ví dụ của đề, và đối chiếu với lời giải tham chiếu viết riêng bằng Python (hoặc bộ kiểm tra đáp án với các bài có nhiều đáp án) trên 60–300 test ngẫu nhiên. Code được viết theo phong cách thi đấu, không chú thích, tên biến đầy đủ, dễ đọc.

Chương 1. Lũy thừa nhanh

Trọng tâm chương

Lũy thừa nhanh tính $a^k \bmod M$ trong $O(\log k)$ bằng cách bình phương liên tiếp. Đây là viên gạch đầu tiên của số học modulo: mọi bài trong tập đều dùng nó. Chương này cũng cho thấy hai ý tưởng đi kèm: đếm theo số mũ của từng số nguyên tố (công thức Legendre), và lũy thừa ma trận để tính nhanh dãy truy hồi tuyến tính.

Kiến thức cần nhớ

- Lũy thừa nhanh. a^k: duyệt các bit của k, bình phương a ở mỗi bước, nhân vào kết quả khi bit bằng 1.
- Tràn số. Đưa mọi thừa số về $[0, M)$ trước khi nhân; với $M \approx 10^9$ tích vừa long long, với $M \approx 10^{18}$ dùng __int128.
- Công thức Legendre. Số mũ của p trong n! là $\lfloor n/p \rfloor + \lfloor n/p^2 \rfloor + \lfloor n/p^3 \rfloor + \dots$
- Lũy thừa ma trận. Truy hồi tuyến tính bậc k tính tới số hạng thứ n trong $O(k^3 \log n)$.
- Dãy dạng tích. Lấy số mũ để chuyển phép nhân thành phép cộng.

Các bài trong chương

Bài	Tên bài	Mã Codeforces	Kĩ thuật chính
1	AND bằng 0, tổng lớn	1514B	Lũy thừa nhanh
2	Số nguyên tố và phép nhân	1238C	Legendre + lũy thừa nhanh
3	Dãy truy hồi dạng tích	1182E	Ma trận + số mũ mod p - 1

Lê Thành Nam

Bài 1. AND bằng 0, tổng lớn

Codeforces 1514B – AND 0, Sum Big Thời gian: 2 giây · Bộ nhớ: 256 MB
Đề gốc: https://codeforces.com/problemset/problem/1514/B

Đề bài

Cho hai số nguyên n và k. Hãy đếm số mảng gồm n phần tử thỏa mãn đồng thời:

- mọi phần tử là số nguyên trong đoạn $[0, 2^k - 1]$;
- phép AND theo bit của tất cả các phần tử bằng 0;
- tổng các phần tử lớn nhất có thể.

Vì kết quả có thể rất lớn, hãy in nó theo modulo $10^9 + 7$.

Dữ liệu vào

Dòng đầu chứa số test t.

Mỗi test là một dòng chứa hai số nguyên n và k.

Ràng buộc chung: $1 \le n, k$.

Kết quả

Với mỗi test, in số mảng thỏa mãn theo modulo $10^9 + 7$.

Ví dụ

Dữ liệu vào	Kết quả
2 2 2 100000 20	4 226732710

Giải thích ví dụ

Với n = 2, k = 2 có 4 mảng: [3, 0], [0, 3], [1, 2] và [2, 1].

Subtask

Subtask	Ràng buộc	Điểm
Subtask 1	n, k ≤ 3	30
Subtask 2	t ≤ 10; n ≤ 10^5; k ≤ 20 (ràng buộc gốc)	30
Subtask 3	t ≤ 10^5; n, k ≤ 10^{18}	40

Lê Thành Nam

Hướng dẫn giải

NGÂY THƠ Thử mọi mảng

Sinh mọi mảng n phần tử, mỗi phần tử từ 0 tới $2^k - 1$, bằng đệ quy. Với mỗi mảng, tính phép AND của các phần tử và tổng; trong các mảng có AND bằng 0, đếm số mảng có tổng lớn nhất.

Code. Tầng 1. Thử mọi mảng

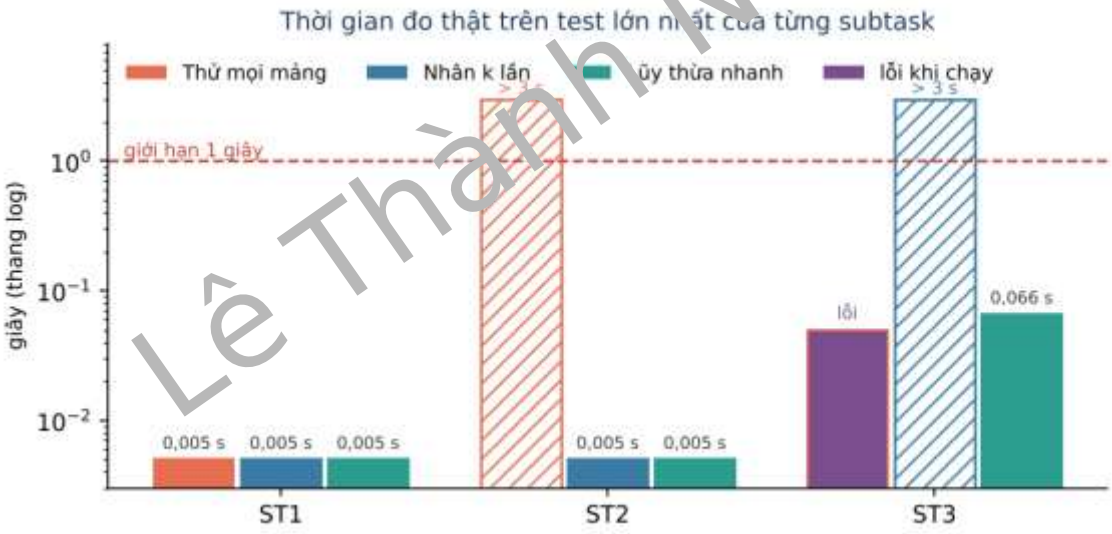
```
#include <bits/stdc++.h>
using namespace std;
long long n, k, limit, bestSum, countBest;
void generate(long long position, long long andAll, long long sum) {
    if (position == n) {
        if (andAll != 0) return;
```

```
        if (sum > bestSum) { bestSum = sum; countBest = 0; }
        if (sum == bestSum) countBest++;
        return;
    }
    for (long long value = 0; value < limit; value++) generate(position + 1, andAll &
value, sum + value);
}
int main() {
    int t;
    scanf("%d", &t);
    while (t--) {
        scanf("%lld %lld", &n, &k);
        limit = k >= 62 ? LLONG_MAX : (1LL << k);
        bestSum = -1;
        countBest = 0;
        generate(0, limit - 1, 0);
        printf("%lld\n", countBest % 1000000007);
    }
}
```

Độ phức tạp: $O((2^k)^n \cdot n)$.

BẾ TẮC $(2^k)^n$ mảng

Với $n = 10^5$, $k = 20$, số mảng là (2^{20}) mũ 10^5: Không thể duyệt (Hình 1.1). Với n tới 10^{18}, đệ quy sâu n tầng còn làm tràn ngăn xếp (ô "lỗi"). Câu hỏi then chốt: Xét riêng từng bit. Điều kiện "AND bằng 0" và "tổng lớn nhất" nói gì về các bit thứ j của n phần tử?



Hình 1.1. *Thời gian đo thật của ba tầng lời giải.*

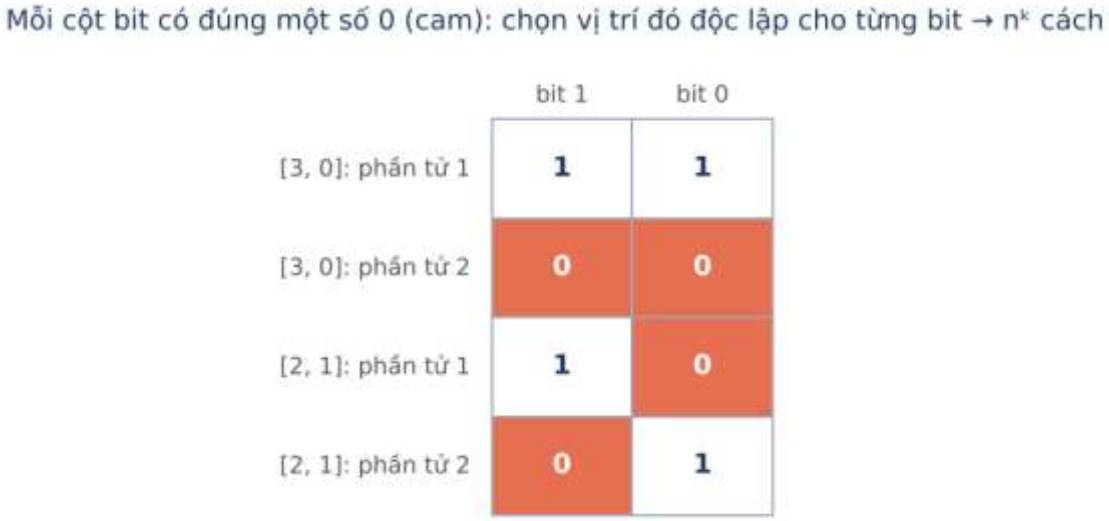
PHÁ BẾ TẮC Phá bế tắc: Mỗi bit có đúng một số 0

AND bằng 0 nghĩa là ở mỗi bit j, có ít nhất một phần tử mang bit 0. Muốn tổng lớn nhất thì mọi bit còn lại phải bằng 1: ở mỗi bit, có đúng một phần tử mang bit 0 (Hình 1.2).

Vậy một mảng tối ưu ứng với việc chọn, cho mỗi bit trong k bit, phần tử nào mang số 0 ở bit đó. Các bit chọn độc lập, mỗi bit n cách: Đáp án là n^k.

Tính n^k bằng cách nhân k lần: Với $k \le 20$ rất nhanh. Nhưng với k tới 10^{18} thì vòng lặp k lần không thể chạy hết.

Lê Thành Nam



Hình 1.2. *Hai mảng tối ưu khi n = 2, k = 2.*

Code. Tầng 2. Nhân k lần

```
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1000000007;
int main() {
    int t;
    scanf("%d", &t);
    while (t--) {
        long long n, k;
        scanf("%lld %lld", &n, &k);
        long long base = n % MOD, answer = 1;
        for (long long i = 0; i < k; i++) answer = answer * base % MOD;
        printf("%lld\n", answer);
    }
}
```

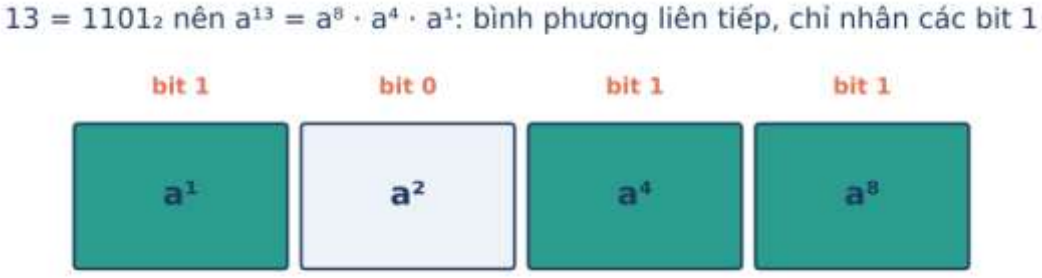
Lê Thành Nam

TỐI ƯU Lũy thừa nhanh

Viết số mũ dưới dạng nhị phân. Ví dụ $13 = 1101_2$ nên $a^{13} = a^8 \cdot a^4 \cdot a^1$ (Hình 1.3). Các lũy thừa a^1, a^2, a^4, a^8, … có được bằng cách bình phương liên tiếp; ta chỉ nhân vào kết quả những lũy thừa ứng với bit 1 của số mũ.

Mỗi vòng lặp xử lí một bit của số mũ, nên chỉ cần khoảng $\log_2 k \approx 60$ vòng với $k \le 10^{18}$. Nhớ lấy n mod $(10^9 + 7)$ trước, vì n cũng tới 10^{18} và tích của hai số như vậy tràn long long.

Với 10^5 test, n, k tới 10^{18}, chạy 0,07 giây.



Hình 1.3. *Lũy thừa nhanh: $a^{13} = a^8 \cdot a^4 \cdot a^1$.*